GCA Session 3 - Lab Report

Stijn Vandendriessche
Master of Electronics and ICT Engineering Technology

Abstract: This report investigates the fundamental concepts of striding, warps, and divergence in GPU computing using CUDA in C++. Striding refers to memory access patterns, warps are groups of threads that are scheduled together, and divergence occurs when threads within a warp take different execution times.
Keywords: CUDA, GPU, CPU, C++

1 Concepts

This report discusses three distinct subjects: warps, striding and divergence.

1.1 Warps

A warp is a group of 32 threads within a block, all of which are scheduled and executed simultaneously on a GPU. Even if the block size is not a multiple of 32, dummy threads are added to form complete warps for scheduling. It's important to note that a warp always consists of threads from only one block, so threads from different blocks are never grouped together in the same warp. To summarize, a grid consists of blocks, a block consists of warps and a warp consists of threads.

1.2 Striding

Striding is a strategy employed when the number of available threads is fewer than the program's input size. Every thread will handle multiple elements. After each iteration is incremented by the total number of threads. Typically, this process is repeated in a while loop until all elements have been processed.

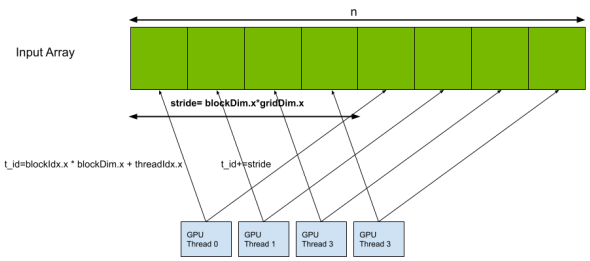


Figure 1: *visualization of GPU striding*

1.3 Divergence

Thread divergence occurs when threads within the same warp have different execution times. This leads to faster threads being idle, waiting for slower threads in their warp to finish. It is good practice to consider this phenomenon when developing GPU applications and try to minimize thread divergence to maximize performance.

2 Project

In order to illustrate the concepts discussed above, I implemented three kernels with a similar task: inverting the colors of an image.

The first inversion kernel uses a loop to try out different gridsizes and blocksizes, to illustrate how warps have a big influence on the execution time if you don't choose these parameters carefully.

The second inversion kernel used the theoretical best values for the GPU in my machine: the NVIDIA GTX 2080 Super with max-Q Design.

The third kernel tries to illustrate divergence by introducing an asymmetry to the problem. In addition to inverting the image, it also modifies the red value.

3 Different gridsizes and blocksizes

The following graph illustrates that different values for gridsize and blocksize lead to verry different execution times. We tried four different gridsizes, with blocks ranging from 2 to 32 threads per block.

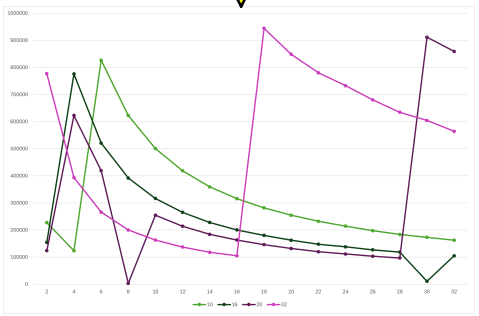


Figure 2: *average execution time per block size for different grid sizes*

4 Theoretical best

My machine has an NVIDIA GTX 2080 Super with max-Q Design. This GPU has 48 SM units, and 3072 cores in total. This means that each SM unit has 64 cores, and will be able to handle a maximum of 2 warps at a time. There are two theoretical best choises for gridsize and blocksize.

- Gridsize of 96
  - Each block consists of 32 threads or 1 warp
- Gridsize of 48
  - Each block consists of 64 threads or 2 warps

5 Github link

`https://github.com/StijnVandendriessche1/GC_lab3.git`